

ALISSON GABRIEL DOS SANTOS ALVES

**: Repurposing de tablets legados via debloat
automatizado para leitura focada**

Salvador

2026

ALISSON GABRIEL DOS SANTOS ALVES

**: Repurposing de tablets legados via debloat automatizado
para leitura focada**

Trabalho de Conclusão de Curso apresentado
ao Colégio Estadual de Aplicação de Tempo
Integral Anísio Teixeira - CEAAT, como parte
dos requisitos para obtenção do Certificado
de conclusão de curso.

Curso: Técnico em Informática

Orientador: Gilberto Monteiro

Salvador
Ano 2026

Resumo

ALVES, Alisson Gabriel dos Santos. : **Repurposing de tablets legados via debloat automatizado para leitura focada**. 2026. Trabalho de Conclusão de Curso (TCC) – Colégio Estadual de Aplicação de Tempo Integral Anísio Teixeira - CEAAT, Salvador, 2026.

Analisa o impacto do padrão de consumo linear na indústria de hardware mobile e o fenômeno do envelhecimento de software (*software aging*), que gera descartes precoces de dispositivos funcionalmente íntegros. Define como objetivo central o desenvolvimento de uma solução de software para promover a preservação ambiental e a saúde digital por meio da ressignificação (*repurposing*) de tablets Android legados em dispositivos de leitura focada (E-readers). Utiliza como objeto de estudo o tablet Multilaser M8 4G da rede pública estadual da Bahia. Emprega uma metodologia aplicada e experimental, fundamentada no uso da ferramenta Android Debug Bridge (ADB) para intervenções não invasivas em três camadas sistêmicas: aplicação, framework e kernel. O método compreende a execução de um *debloat* radical para remoção de serviços redundantes, o ajuste de parâmetros ergonômicos como escala de cinza e densidade de pixels para mitigar a Síndrome da Visão do Computador (CVS), além da aplicação de *underclock* e redução da taxa de atualização da tela para otimização energética. Valida a eficácia da intervenção por meio de análise quantitativa de desempenho e autonomia, utilizando métricas de *Throughput*, *Slowdown* e o modelo de Desejabilidade Composta para equilibrar performance e consumo. Conclui que a engenharia de software consciente permite a conversão de potenciais resíduos eletrônicos em ativos educacionais valiosos, comprovando a viabilidade técnica de políticas de TI sustentáveis e a democratização do acesso a ferramentas de leitura fluida e livre de distrações.

Palavras-chave: Debloat Android. Green IT. Saúde Digital. Obsolescência de Software. Deep Work.

Abstract

ALVES, Alisson Gabriel dos Santos. : **Repurposing legacy tablets via automated debloat for focused reading**. 2026. Graduation Thesis (TCC) – Colégio Estadual de Aplicação de Tempo Integral Anísio Teixeira - CEAAT, Salvador, 2026.

This research analyzes the impact of linear consumption patterns in the mobile hardware industry and the phenomenon of software aging, which triggers the premature disposal of functionally sound devices. The central objective is the development of a software solution that combines environmental preservation and digital health by repurposing legacy Android tablets into focused reading devices (E-readers). The study specifically focuses on the Multilaser M8 4G tablet distributed within the Bahia state public school system. The methodology is classified as applied and experimental, utilizing a mixed approach to validate hardware repurposing. The technical procedure employs the Android Debug Bridge (ADB) for non-invasive interventions across three systemic layers: application, framework, and kernel. The method involves automated debloating to remove redundant services and Google Mobile Services (GMS), the adjustment of ergonomic parameters such as grayscale and pixel density to mitigate Computer Vision Syndrome (CVS), and kernel-level underclocking to maximize energy efficiency. The effectiveness of the intervention is validated through quantitative performance and autonomy analysis, using Throughput and Slowdown metrics alongside Composite Desirability models to balance performance and power consumption. It concludes that conscious software engineering enables the conversion of potential electronic waste into valuable educational assets, proving the technical viability of sustainable IT policies and the democratization of distraction-free reading tools.

Keywords: Android Debloat. Green IT. Digital Health. Software Obsolescence. Deep Work.

Sumário

1	INTRODUÇÃO	5
1.1	Problema de Pesquisa	5
1.2	Objetivos	5
2	JUSTIFICATIVA	7
3	METODOLOGIA	8
4	DESENVOLVIMENTO E IMPLEMENTAÇÃO	10
4.1	Intervenções em Nível de Aplicação	10
4.1.1	Configuração do Ambiente e Protocolo de Comunicação	10
4.1.2	Lógica de Automação e Processamento de Bloatwares	11
5	CONSIDERAÇÕES FINAIS E RESULTADOS ESPERADOS	13
	REFERÊNCIAS	14

1 INTRODUÇÃO

A crise ambiental contemporânea é considerada por muitos o problema de maior relevância da atualidade, sendo essa, muito impulsionada pelo atual padrão de consumo linear da indústria no geral. Esse padrão se mostra ainda mais forte no contexto de *hardwares mobile*, que são renovados a cada ano, prometendo grandes mudanças, deixando em pouco tempo aparelhos funcionais obsoletos ou "artificialmente incapazes", Parnas (1994) diz que esse efeito está fundamentalmente ligado ao que ele chama de *software aging* onde a medida que as aplicações se atualizam elas exigem cada vez mais do hardware, ainda que o mesmo seja perfeitamente capaz tornando o software em um "fardo oneroso". Essa prática impulsiona o aumento da geração de resíduos sólidos, em especial, o lixo eletroeletrônico, que possuem metais pesados de difícil decomposição. Segundo o relatório The Global E-waste Monitor (BALDé et al., 2024), a geração de lixo eletrônico, estimada em 62 bilhões de kg no ano de 2022, cresce cinco vezes mais do que a reciclagem documentada, com apenas 22,3% dos resíduos coletados adequadamente em 2022.

1.1 Problema de Pesquisa

A "incapacidade artificial" nos dispositivos móveis é impulsionada pelo *overhead* de processamento do Google Mobile Services (GMS) e pela evolução das camadas de *runtime* (ART) e pelas atualizações do sistema que vêm perfeitamente otimizadas para o hardware de última geração, mas com grandes falhas de otimização para os mais antigos. Isso deixa dispositivos com hardware perfeitamente funcionais, freitados pelo software.

1.2 Objetivos

A presente pesquisa tem por objetivo o desenvolvimento de uma solução de software que combine preservação ambiental e saúde digital através da *repurposing* de tablets Android, em E-readers, dando um novo significado funcional para dispositivos obsoletos. Em específico o tablet Multilaser M8 4G distribuído pelas escolas da rede pública estadual da Bahia.

Nesse dispositivo, será realizado um *debloat* automatizado para otimizar o uso do armazenamento, memória RAM e ciclos de CPU, junto a implementação de uma interface monocromática e ajustes ergonômicos para redução de fadiga visual. Tal preocupação com a fadiga visual e ergonomia, vem do crescente índice de CVS (Síndrome da Visão do Computador), a qual atinge cerca de 70% da população mundial (Sá, 2016). Além disso, outra preocupação da presente pesquisa é a validação da autonomia energética, conforme

Monteiro et al. (2023) o perfil do usuário e serviços de fundo são os maiores vilões da bateria. A automatização também terá como meta o ajuste de parâmetros do Kernel para maximizar a autonomia energética. Essa automação será disponibilizada de forma Open Source em portais como Github de forma gratuita e em linguagens como Shell Script (Bash) e Python.

2 JUSTIFICATIVA

A necessidade dessa pesquisa é fortemente fundamentada na urgência de combater o modelo de consumo linear consolidado atualmente. Em 2022, a categoria de resíduos sólidos, que inclui aparelhos eletrônicos como celulares, tablets e laptops, chegou a acumular 5 bilhões de kg de lixo (BALDÉ et al., 2024). Entretanto, o destino da maioria dos dispositivos obsoletos, não é o descarte, mas sim a chamada hibernação doméstica. Estima-se que famílias da União Europeia possuam cerca de 74 itens eletrônicos em suas casas, dos quais, boa parcela está quebrada ou sem uso. No Brasil, a Associação Brasileira de Reciclagem de Eletroeletrônicos e Eletrodomésticos (ABREE, 2024) também sinaliza que cerca de 85% dos brasileiros possuem algum tipo de aparelho eletrônico parado em suas casas. O Brasil também é o maior gerador de *E-waste* da América Latina segundo a ABREE (2024). Esses dados representam um panorama preocupante em que por diversos fatores, dispositivos legado em seu pleno funcionamento são encontrados no seu estado de hibernação. A presente pesquisa pode ajudar a mitigar os danos dessa situação, dando um novo propósito para aqueles dispositivos que já não servem bem as exigências dos softwares e aplicativos modernos.

Somado a isso, o desenvolvimento da pesquisa também ancora-se nos conceitos de saúde digital e minimalismo para promover o *Deep Work* por meio de modificações em várias das partes que compõem o sistema Android. De acordo com Newport (2019), a tecnologia dos dias de hoje é projetada para capturar nossa atenção, o que, em dispositivos de múltiplas funções mostra-se um agente de fragmentação do foco, com diversos serviços disputando pelo seu olhar e atrapalhando o chamado *Deep Work*. A proposta de ressignificação visa também combater essa natureza de modo a tornar o aparelho em um dispositivo de leitura livre de distrações.

Finalmente, o projeto possui um forte componente de conveniência e democratização tecnológica. Ao transformar um dispositivo comum, como Multilaser M8 4G da rede estadual, em uma ferramenta educacional fluida, o estudo remove a barreira financeira imposta pelo alto custo de dispositivos especializados como o Kindle. Trata-se de uma prova de conceito de que a Engenharia Verde pode converter um dispositivo que seria trocado e descartado, em um ativo educacional valioso e acessível.

3 METODOLOGIA

A presente pesquisa classifica-se como aplicada e experimental, com abordagem mista para validar a eficácia da ressignificação do hardware. O procedimento técnico central vai utilizar o Android Debug Bridge (ADB), como principal ferramenta, permitindo a comunicação entre a automação e o tablet de forma nativa e segura. Trata-se de uma ferramenta de CLI (*Command Line Interface*) oficial de desenvolvimento disponibilizada pelo Android Open Source Project (AOSP). Operando no modelo cliente-servidor, ela dá acesso ao Shell Unix dos dispositivos Android de forma não invasiva e sem necessidade de acesso root (ANDROID DEVELOPERS, 2025). Essa ferramenta possibilita a escrita de um código para realizar alterações profundas nos dispositivos, essas que vão acontecer em 3 camadas principais:

1. **Nível de Aplicação:** Nesta etapa o script vai, de forma automática, instalar os aplicativos necessários para o pleno funcionamento do dispositivo como um E-reader, e remover processos redundantes ou aplicativos pré-instalados (*bloatwares*), fazendo um *debloat* completo. Desse modo, além de remover redes sociais, apps que prejudicam o *deep work* e atrapalham o foco, também serão removidos, caso o usuário optar, os chamados Google Mobile Services (GMS) que incluem telemetrias intrusivas. Esta ação libera recursos desperdiçados em coisas desnecessárias e constroem a base para as modificações mais profundas.
2. **Nível de Framework:** Essa é a fase em que ocorre a manipulação do banco de dados `settings.db` e do gestor de janelas (*Window Manager* ou WM) para alterar diversas das configurações padrão. Nessa fase, algumas das principais modificações vão ser a aplicação de um filtro de luz azul nativo, simulando E-Inks de E-reader especializados. Essa alteração mostra-se relevante pois, segundo Benedetto et al. (2013), as telas E-ink diminuem drasticamente o risco de CSV, e embora a tela continue sendo LCD, a redução da luz azul e a escala de cinza reduzem o brilho e o contraste agressivo, mitigando a CVS, embora não atinja as propriedades físicas de reflexão de luz do papel eletrônico real aumentará o conforto ocular do usuário. A manipulação da densidade de pixels (DPI) e da escala de cinza também será aplicada visando otimizar a renderização de fontes, reduzindo a carga cognitiva e visual durante a leitura (Sá, 2016; ANDROID DEVELOPERS, 2025).
3. **Nível de Kernel:** Por fim, essa é a parte de maior dificuldade, mas que vai aplicar as maiores diferenças práticas. Aqui, a automação vai aplicar técnicas de *underclock* (limitação da frequência do processador) e redução da taxa de atualização da tela

via Surface Flinger, para tornar a tela do aparelho mais semelhante ao padrão da indústria, além de ajudar com a eficiência energética.

Essas modificações serão acompanhadas de uma análise de dados para medir a diferença de desempenho, uso de recursos de hardware e autonomia da bateria, por meio de uma comparação quantitativa usando métricas como *Throughput* e *Slowdown* para comprovar se a alteração foi de fato benéfica ou não do ponto de vista de fluidez do sistema, utilizando modelos de Desejabilidade Composta para equilibrar desempenho e consumo energético (PONTES, 2017).

4 Desenvolvimento e Implementação

O presente capítulo, descreve os procedimentos técnicos realizados para a elaboração da solução, estruturando-se conforme os níveis sistêmicos estabelecidos na metodologia: Aplicação, framework e Kernel. A implementação foi conduzida em ambiente desktop Linux, visando integração nativa com ferramentas de automação via terminal.

4.1 Intervenções em Nível de Aplicação

O primeiro nível de intervenção foca na desoneração do sistema através da remoção de processos e serviços irrelevantes para o produto final, bem como a integração das ferramentas necessárias para estabelecer uma experiência de leitura focada. Entretanto, antes de iniciar-se a etapa de desenvolvimento prático, faz-se necessário a preparação do ambiente e das ferramentas que vão permitir a comunicação entre os dispositivos.

4.1.1 Configuração do Ambiente e Protocolo de Comunicação

O desenvolvimento se inicia pela instalação e preparação da ferramenta *Android Debug Bridge* (ADB), que, em consonância com a metodologia estabelecida, trata-se de uma ferramenta de linha de comando (*Command Line Interface - CLI*) oficial do AOSP (ANDROID DEVELOPERS, 2025), que será responsável por cada um dos comandos e modificações realizadas ao longo do processo. A instalação do ADB, em especial em ambientes Linux desktop caracteriza-se pela simplicidade, se resumindo a um único comando do gerenciador de pacotes da distribuição. Contudo, apesar do rápido providenciamento, a ferramenta possui uma arquitetura robusta e abrangente.

Em termos funcionais, o ADB opera em uma arquitetura de cliente-servidor (TANENBAUM; BOS, 2016), o qual permite a troca de informações e o envio de instruções entre os dois dispositivos, nessa estrutura, o servidor (executado no tablet) atua de forma reativa, processando requisições e retornando dados, enquanto o cliente (executado na estação de trabalho) detém a prerrogativa de ordenar as execuções. Paralelamente, estabeleceu-se o controle de versão via repositório Git, permitindo o rastreamento de modificações e a organização colaborativa do código-fonte, parte crucial do projeto visto que o mesmo tem por objetivo se manter *open source*¹. A manutenção do código em formato aberto (*open source*) justifica-se pela necessidade de auditabilidade e transparência, sobretudo em um projeto que realiza modificações profundas no sistema operacional.

¹ Open Source é uma forma de manter projetos de software onde o autor disponibiliza todo seu código (source) publicamente para que outras pessoas ou empresas possam usar ou contribuir, desde que sigam as regras da sua licença

No entanto, por conceder acesso ao *shell Unix*² do celular, a abertura desse protocolo pode se tornar um vetor de invasão preocupante sem as devidas proteções. Para mitigar tais riscos, o sistema Android implementa medidas de segurança rigorosas fundamentadas em permissões de usuário, sandboxes e na autenticação por chaves de criptografia assimétrica RSA. No ecossistema Android cada um dos aplicativos roda em um ambiente isolado e tem um identificador de usuário único e exclusivo (UID) com permissões diferentes. Isso garante que, por padrão, nenhum programa tenha acesso a dados dos outros, nem ao sistema. Embora o Android, diferente de outros sistemas, não disponibilize um usuário root (administrador) por padrão, os comandos do ADB são executados por um usuário especial com permissões elevadas chamado "shell". Esse usuário pode gerenciar permissões de outros, ler *logs*³ de sistema, alterar configurações internas e desinstalar e instalar programas.

Para mitigar acessos não autorizados a esse usuário privilegiado, o protocolo utiliza de a criptografia assimétrica. Até a versão 4.2.2 do Android bastava ativar a depuração e qualquer máquina poderia executar comandos no *shell*, criando uma vulnerabilidade como a adulteração de aparelhos furtados, ou ataques por meio de estações de carregamento maliciosas (*juice jacking*) em locais públicos. Para corrigir isso o Google implementou uma autenticação por chave RSA, que funciona com uma chave pública (`adbkey.pub`) e uma privada (`adbkey`). Quando você conecta um computador, o celular pergunta se deve dar permissão para aquele computador, e, ao tocar em "Sempre permitir a partir desse computador", uma cópia da chave pública é salva no celular para validar as conexões futuras e garantir a integridade da automação sem violar os protocolos de segurança nativos do sistema.

4.1.2 Lógica de Automação e Processamento de Bloatwares

Consolidados o ambiente de desenvolvimento e repositório Git, o desenvolvimento da automação inicia-se através de uma verificação inteligente dos aparelhos servidores conectados. Essa medida visa assegurar que o *script* execute apenas quando houver exatamente um tablet conectado ao barramento USB, evitando possíveis conflitos entre os comandos, assim como previne que a automação rode sem nenhum dispositivo conectado.

Validada a conexão, a próxima fase seria a desoneração do sistema via remoção de softwares prescindíveis (debloat). A desinstalação de pacotes do dispositivo não se restringe a uma otimização superficial, mas constitui uma estratégia para conter o envelhecimento de software (software aging). Conforme fundamentado por Parnas (PARNAS, 1994), o envelhecimento de software ocorre quando o programa se torna um "fardo oneroso" devido

² Unix foi um dos primeiros e mais importantes sistemas operacionais, estabelecendo padrões usados até hoje inclusive no Android, Linux e MacOS.

³ Logs são registros do que acontece em tempo de execução de uma aplicação, usado para monitoramento e diagnósticos

ao acúmulo de atualizações e serviços contínuos, esses que são voltados para o hardware mais novo, mas também são empurrados para o dispositivo legado, demandando ainda mais recursos computacionais. Ao eliminar o excesso de processos, a automação restaura a fluidez do aparelho e diminui os agentes de distração, permitindo a execução de tarefas simples como a leitura, serem mais confortáveis e principalmente convenientes.

Para estruturar o *debloat* sem poluir o código principal com chamadas massantes, foi adotado uma abordagem declarativa baseada no arquivo de configuração `blacklist.json`. Esse arquivo lista os nomes exatos de pacotes de rede sociais, ferramentas do *Google Mobile Services* e demais *bloatwares* identificados como prejudiciais ao foco ou ao desempenho do sistema. A escolha da remoção do GMS fundamenta-se na eficiência energética: estudos de mapeamento sistemático indicam que aplicativos e serviços que operam em segundo plano são principais responsáveis pela drenagem de energia em dispositivos Android (MONTEIRO et al., 2023). A eliminação desse tipo de tarefa, previne que o dispositivo acorde da hibernação por motivos irrelevantes, maximizando a autonomia em textitidle.

O formato JSON confere flexibilidade ao projeto por sua sintaxe de fácil leitura e modificação, permitindo ao usuário adicionar ou remover *apps* da lista se necessário com facilidade. Sob a perspectiva conceitual, essa modularidade alinha-se aos preceitos do Minimalismo Digital (NEWPORT, 2019), o qual preconiza que a tecnologia deve ser configurada de forma seletiva e intencional.

Porém, para o script ler e manipular-lo de forma efetiva, foi necessário a integração da ferramenta de linha de comando "jq", foi implementada em uma estrutura de repetição que percorre toda a lista e executa uma tentativa de desinstalação para cada objetos. Em adição, o *looping* também salva um arquivo de **log** com o resultado de cada um dos comandos, o que permite a auditoria do processo e validação da execução. A intenção é que todas as camadas salvem seus resultados no arquivo de log.

Ao término do *debloat*, a camada de aplicação já está quase completa, restam agora a instalação das ferramentas que serão essenciais no uso do produto final. Planeja-se baixar diretamente de lojas open-source como *F-droid*, através de ferramentas como *curl* ou *wget*, e a possível troca do launcher, visando minimalismo e diminuição dos vetores de distração. Do âmbito da ergonomia cognitiva do usuário, removendo elementos gráficos apelativos e notificações projetadas para reter atenção.

5 CONSIDERAÇÕES FINAIS E RESULTADOS ESPERADOS

Espera-se demonstrar que os dispositivos legados ainda possuem alto potencial técnico quando desonerados de softwares mal otimizados, podendo ser adaptados para tarefas que não demandam alto desempenho, assim evitando o descarte. O artefato final servirá como prova de conceito para políticas de TI sustentáveis, provando que a engenharia de software consciente pode converter o que seria resíduo eletrônico em um ativo educacional valioso e focado no ser humano.

Referências

ABREE. *Relatório de Resultados de Logística Reversa de Eletroeletrônicos e suas Embalagens 2023-2024*. São Paulo, 2024. 7

ANDROID DEVELOPERS. *Android Debug Bridge (adb)*. 2025. Disponível em: <<https://developer.android.com/tools/adb>>. Acesso em: 1 ago. 2026. 8, 10

BALDÉ, C. P. et al. *The Global E-waste Monitor 2024: Quantities, flows and the global e-waste challenge*. Geneva/Bonn, 2024. 5, 7

BENEDETTO, S. et al. E-readers and visual fatigue: Comparison between e-ink and lcd. *PLOS ONE*, v. 8, n. 12, p. e83676, 2013. 8

MONTEIRO, E. et al. Analysis of energy consumption on android devices for developers: A systematic mapping study. *SBC Reviews on Computer Science*, v. 29, n. 1, 2023. 6, 12

NEWPORT, C. *Minimalismo Digital: para uma vida profunda em um mundo superficial*. Rio de Janeiro: Alta Books, 2019. 7, 12

PARNAS, D. L. Software aging. In: *Proceedings of the 16th International Conference on Software Engineering*. Hamilton: IEEE, 1994. p. 279–287. 5, 11

PONTES, J. C. N. *Análise de desempenho e consumo de energia de parâmetros TCP/IP em dispositivos móveis*. 71 p. Dissertação (Mestrado) — Centro de Informática, Universidade Federal de Pernambuco, Recife, 2017. 9

Sá, E. C. *Síndrome da visão do computador e função visual em trabalhadores usuários de computador de um hospital público universitário de São Paulo: prevalência e fatores associados*. 107 p. Tese (Doutorado) — Faculdade de Saúde Pública, Universidade de São Paulo, São Paulo, 2016. 5, 8

TANENBAUM, A. S.; BOS, H. *Sistemas Operacionais Modernos*. 4. ed. São Paulo: Pearson Education do Brasil, 2016. 10